

Step 5 - Workspace + Shared Utils

Phase B · Folder Discipline · %run · One Source of Truth · ~20 min

Goal: Professional workspace organization + a shared masking-functions notebook proven via %run. One source of truth for compliance-critical code. Artifacts: masking_functions + utils_test notebooks.

LEARN

Why a utils notebook matters: SIN-masking is compliance-critical code. Copy-pasted across five notebooks, one buggy copy = a PIPEDA problem with no single fix point. Banks keep masking/validation in ONE shared notebook. When the auditor says 'show me your masking implementation', you show one file.

%run executes another notebook inline - its functions become available. It must be ALONE in its own cell or it silently fails.

Folder discipline: production notebooks in shared organized folders; experiments in user folders; nothing personal ever gets scheduled.

LAB

Part 1 — Folders: create /MapleBank/ with subfolders bronze, silver, gold, utils; move notebooks 01-04 in.

Part 2 — The utils notebook (MapleBank/utils/masking_functions):

```

# MapleBank Shared Utilities - PII Masking & Validation
# ONE SOURCE OF TRUTH (PIPEDA/OSFI). Usage: %run ../utils/masking_functions
import pyspark.sql.functions as F

def mask_sin(col_name="sin"):
    """872-948-301 -> ***-***-301"""
    return F.when(F.col(col_name).isNotNull(),
        F.concat(F.lit("***-***-"), F.substring(F.col(col_name), 8, 3))
    ).otherwise(F.lit("***-***-***"))

def mask_account(col_name="account_number"):
    """4859771 -> ****771"""
    return F.when(F.col(col_name).isNotNull(),
        F.concat(F.lit("****"), F.substring(F.col(col_name), -3, 3))
    ).otherwise(F.lit("****"))

def postal_to_fsa(col_name="postal_code"):
    """M5V 2T6 -> M5V (FSA ~ 7,000 people - analytics-safe)"""
    return F.when(F.col(col_name).isNotNull(),
        F.upper(F.substring(F.col(col_name), 1, 3))
    ).otherwise(F.lit("UNK"))

def mask_email(col_name="email"):
    """john.smith@x.ca -> j***@x.ca"""
    return F.when(F.col(col_name).contains "@" ,
        F.concat(F.substring(F.col(col_name), 1, 1), F.lit("***@"),
            F.split(F.col(col_name), "@").getItem(1))
    ).otherwise(F.lit("***"))

def validate_transit(col_name="transit_number"):
    return F.col(col_name).rlike("^[0-9]{5}$")

def validate_province(col_name="province"):
    return F.col(col_name).isin("ON","QC","BC","AB","MB","SK",
        "NS","NB","NL","PE","YT","NT","NU")

def add_audit_columns(df, source_name, business_date):
    """Bronze metadata: source traceability for OSFI audit."""
    return df \
        .withColumn("_source_file", F.lit(source_name)) \
        .withColumn("_ingestion_timestamp", F.current_timestamp()) \
        .withColumn("_business_date", F.lit(business_date))

print("MapleBank utils loaded")

```

Part 3 — Prove %run works (notebook utils_test):

```

# Cell 1 (ALONE in the cell):
%run ../utils/masking_functions

# Cell 2:
VOL = "/Volumes/workspace/default/maplebank"
df_customer = spark.read.option("header", True).csv(f"{VOL}/dim_customer.csv")
df_masked = df_customer \
    .withColumn("sin_masked", mask_sin("sin")) \
    .withColumn("postal_code_fsa", postal_to_fsa("postal_code")) \
    .withColumn("email_masked", mask_email("email")) \
    .withColumn("province_valid", validate_province("province"))
df_masked.select("customer_id", "sin", "sin_masked", "postal_code",
    "postal_code_fsa", "email_masked", "province_valid").show(5, truncate=False)

```

COMMIT + CHECKPOINT

Export both notebooks to databricks/ on GitHub. Pass criteria: masked columns appear beside originals, with functions sourced from the OTHER notebook. If %run fails: check it is alone in its cell, and verify the path (notebook ... menu - Copy path).

